

Django Admin Panel

1. Introduction: What is the Admin Panel?

Before diving into code, it helps to frame why this tool exists. Students often confuse the admin panel with the front-end user experience.

The Django Admin is a built-in, dynamically generated backend interface. Unlike the front-end of your website (which your end-users see), the admin panel is a trusted environment for site administrators and staff to perform CRUD (Create, Read, Update, Delete) operations on the database.

It reads the metadata from your Python models and automatically constructs a functional, secure graphical interface. It is incredibly powerful for content management, data entry, and internal tooling right out of the box.

The Prerequisites (Recap)

To access the admin interface, you must have completed three steps:

1. **Migrations:** `python manage.py migrate` (Creates the necessary database tables, including auth tables for users).
2. **Superuser Creation:** `python manage.py createsuperuser` (Creates an administrative account).
3. **Running the Server:** Navigate to `http://127.0.0.1:8000/admin/` and log in.

2. Registering a Simple Model

By default, Django does not expose your custom models to the admin interface. You must explicitly register them in your app's `admin.py` file.

Here is how to register the `Post` model so that it appears in the GUI.

blog/admin.py

Python

```
from django.contrib import admin
from .models import Post
```

```
# The simplest way to register a model
admin.site.register(Post)
```

Steps: Refresh the admin page. You will now see the "Post" model listed. Click into it, add a new post, and show them the default form. Django automatically recognizes the status field as a dropdown because of the `choices=STATUS` tuple in the model!

3. Customizing the ModelAdmin Class

The simple registration works, but it usually displays objects generically (e.g., "Post object (1)").

We can drastically improve the user experience by creating a `ModelAdmin` class.

Update `admin.py` to customize which columns are shown, add filtering, and enable search

functionality:
blog/admin.py

Python

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    # Changes the columns shown in the list view
    list_display = ('title', 'author', 'status', 'created_on')

    # Adds a filter sidebar on the right
    list_filter = ('status', 'created_on', 'author')

    # Adds a search bar at the top (searches these specific fields)
    search_fields = ['title', 'content']

    # Automatically populates the slug based on the title (if you add a slug field later)
    # prepopulated_fields = {'slug': ('title',)}
```

Steps: Refresh the post list view. The generic "Post object (1)" is gone, replaced by a neat table showing the title, author, status, and creation date. Demonstrate filtering by "Published" vs "Draft" and using the search bar.

4. Expanding the Data: Adding a Comment Model

Let's add a related model to demonstrate relational data. Add this to your models.py.

blog/models.py

Python

```
class Comment(models.Model):
    # The ForeignKey ties each comment to a specific Post.
    # 'related_name' is how we will access comments from the Post side.
    post = models.ForeignKey(Post, on_delete=models.CASCADE, related_name='comments')
    author_name = models.CharField(max_length=80)
    body = models.TextField()
    created_on = models.DateTimeField(auto_now_add=True)
    approved = models.BooleanField(default=False)

    def __str__(self):
        return f"Comment {self.body} by {self.author_name}"
```

The "Missing Table" Error

See it in action: *Intentionally try to access a view or the admin panel that references this new model right now. You will get a database error.*

The Error: django.db.utils.OperationalError: no such table: blog_comment

Why it happens: You have written the Python code, but the database has no idea this table exists yet. Django is trying to query a table that hasn't been built.

Migrating the New Model

To fix this, translate the Python code into database instructions, then apply them:

1. `python manage.py makemigrations` (*Creates the blueprint/instructions*)
2. `python manage.py migrate` (*Executes the instructions on the database*)

5. Displaying Related Comments in a Template

Before managing comments in the admin panel, let's look at how they are rendered on the front end using the `related_name` we defined in the model.

In your post detail template (e.g., `post_detail.html`), you can loop through the comments connected to that specific post:

post_detail.html

HTML

```
<h2>{{ post.title }}</h2>
<p>{{ post.content }}</p>

<hr>
<h3>Comments:</h3>
{% for comment in post.comments.all %}
  <div class="card mb-3">
    <div class="card-body">
      <h5 class="card-title">{{ comment.author_name }}</h5>
      <p class="card-text">{{ comment.body }}</p>
      <small class="text-muted">Posted on {{ comment.created_on }}</small>
    </div>
  </div>
{% empty %}
  <p>No comments yet. Be the first to comment!</p>
{% endfor %}
```

6. Managing Comments in the Admin Panel

Approach A: The Simple Registration

Just like the Post model, you can register Comment entirely on its own.

blog/admin.py

```
Python
from .models import Post, Comment
```

```
# ... PostAdmin code ...
```

```
admin.site.register(Comment)
```

The result: You get a separate "Comments" section in the admin GUI. This is fine, but structurally disconnected. If you are reading a specific Blog Post, you have to leave that page, go to the Comments section, and filter by the post to see what people are saying about it.

Approach B: Admin Inlines (The Professional Way)

Django provides "Inlines" to solve this. Inlines allow you to edit a parent model (the Post) and its related child models (the Comments) on the exact same page.

Update your admin.py to look like this:

blog/admin.py

```
Python
from django.contrib import admin
from .models import Post, Comment

# 1. Define the Inline class for the child model
class CommentInline(admin.TabularInline): # Alternatively: admin.StackedInline
    model = Comment
    extra = 1 # The number of empty forms to display at the bottom

# 2. Attach the Inline to the parent model's Admin class
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'status', 'created_on')
    list_filter = ('status', 'created_on', 'author')
    search_fields = ['title', 'content']

    # Register the inline here
    inlines = [CommentInline]

# Note: We can remove admin.site.register(Comment) if we only want to
# manage comments directly from the Post page.
```

Steps: Navigate to the admin page for a specific Post. Scroll to the bottom of the page. You will now see a tabular layout of all comments specifically attached to that post, allowing you to edit, approve, or delete comments without ever leaving the blog post's editing screen.